

Introduction to SSD_FrameBuf

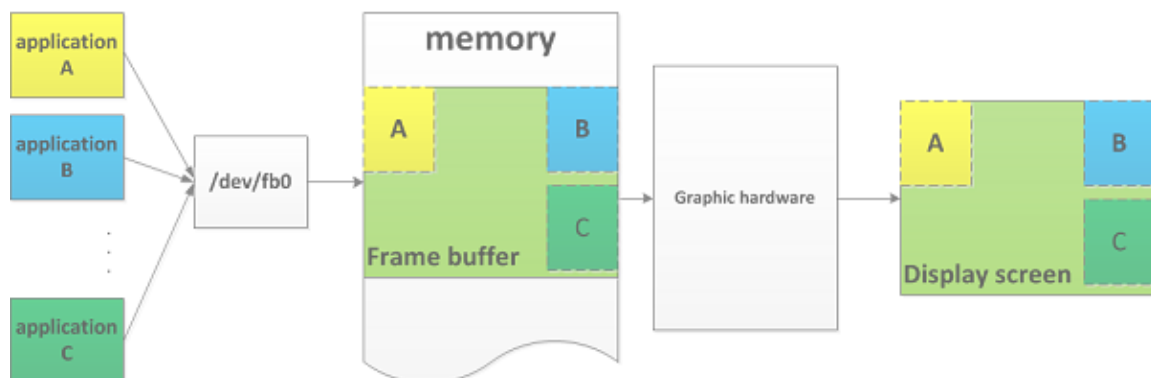
1 Overview

fbdev is used to provide a layer of software abstraction for display graphics hardware. It represents the frame memory of display graphics hardware and provides some well-defined interfaces for application software to access graphics hardware without caring about the specific control details of the underlying graphics hardware. .

Access to fbdev is usually through some specific device nodes, such as `/dev` the `/dev/fb*` .

A simple framebuffer usage scenario:

The three applications modify the framebuffer through the fb device node of fb0, and then the graphics hardware outputs the modified content to the display side



2. User's View of /dev/fb*

fbdev is currently a character device, the first device number is 29, and the tail device number indicates the frame buffer number

For convenience, the device node will be generated as follows (the numbers indicate the device number):

```
0 = / dev / fb0 First frame buffer
1 = / dev / fb1 Second frame buffer
...
31 = / dev / fb31 32nd frame buffer _
```

fbdev is also an ordinary memory device. This means, you can read and write its contents. for example:

1. You can take a screenshot by doing the following:

```
cp /dev/fb0 myfile
```

2. Draw a white pixel to the upper left corner of the screen:

```
echo -en '\xFF\xFF\xFF\x00' > /dev/fb0
```

3. mmap /dev/fb0 for finer drawing (this part will be detailed in other chapters):

```
int fb = open("/dev/fb0", O_RDWR);
assert(fb > 0);
struct fb_var_screeninfo info;
assert(0 == ioctl(fb, FBIOGET_VSCREENINFO, &info));
size_t len = 4 * info.xres * info.yres;
uint32_t *buf = mmap(NULL, len, PROT_READ | PROT_WRITE,
MAP_SHARED, fb, 0);
assert(buf != MAP_FAILED);
```

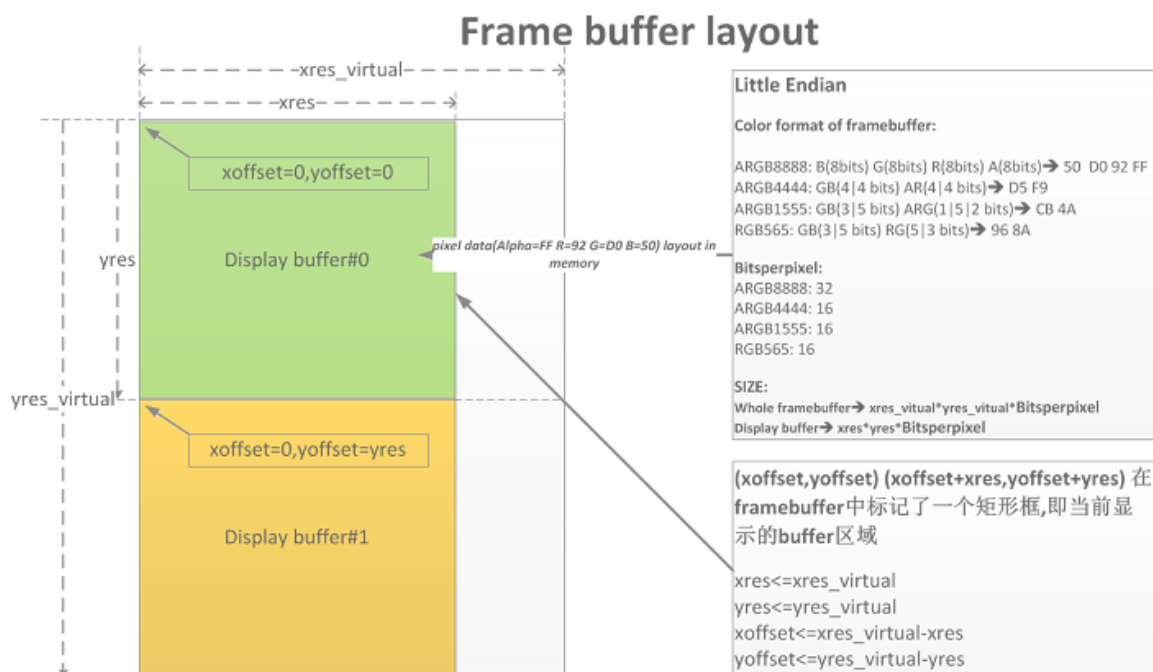
The pixel at (x,y) can be easily modified by assigning a value to buf[y * info.xres + x].

.....

3. Closer look to /dev/fb*

简介提到fbdev代理了图形硬件的显示帧内存，并且提供了一些接口控制这部分内存，对比上文的framebuffer简单应用场景。

下图是framebuffer的memory layout:



- 不同的fbdev的实现，让fbdev代理的图形硬件的显示帧内存 (framebuffer) 会具备不同的初始属性.有些是可配置的属性 (通过ioctl(fb , FBIOGET_VSCREENINFO , &info)获取struct fb_var_screeninfo defined in "linux/fb.h") ，有些则是固定属性 (通过ioctl(fb , FBIOGET_FSCREENINFO , &info)获取struct fb_fix_screeninfo defined in "linux/fb.h") .
- sstar 平台的初始设定方式参见4.1章节 [#jump]

以上图为例我们需要关注几个主要属性：

- 颜色格式(colorformat)

framebuffer的颜色格式可以通过ioctl(fb , FBIOPUT_VSCREENINFO , &info) 设定，通常我们都是初始化fbdev的时候就决定好. 对fbdev的用户来说，确定了颜色格式用户就知道如何修改对应pixel的内容。

- **bitsperpixel**

一个pixel的宽度，与颜色格式协调的变量，颜色格式确定了pixel的宽度也确定了。

- **xres_virtual/yres_virtual**

可通过ioctl FBIOGET_VSCREENINFO/FBIOPUT_VSCREENINFO获取。通常fbdev 代理的framebuffer总大小为:xres_virtual*yres_virtual*bitsperpixel。

- **xres/yres,xoffset/yoffset**

(xoffset,yoffset) (xoffset+xres,yoffset+yres) 在framebuffer中标记了一个矩形框，即当前显示的buffer区域.也就是说display buffer 用于显示的有效区域可以是从小framebuffer开始地址开始。

为何 display buffer 在 framebuffer内

- 支持不同分辨率的内容.

有时候framebuffer的内存大小在FBDEV初始化的时候已经申请好了. 我们可能需要申请一个能够容纳最大分辨率的framebuffer，比如1080p，此时我们使用的display buffer 可能是720p.

- **double buffer.**

使用单buffer无法避免画面撕裂的问题，因为你可能总是修改未显示的内容，造成已经被graphic hardware显示的部分和被修改将要显示的部分时空错位。使用双buffer，显示一张display buffer#0，修改另一张显示一张display buffer#1，修改完成之后交换.显示一张display buffer#1，修改另一张显示一张display buffer#0.

4. sstar平台 FBDEV介绍

4.1. 初始化

- 实现FBDEV的软件模块fbdev.ko。
- 以该方式加载FBDEV:insmod fbdev.ko [default_config_path_file=fbdev configure file(absolute path)]，配置文件可选，系统默认的加载的配置文件路

径 /config/fbdev.ini 。

4.2. 配置文件fbdev.ini

```
# FBDEV 将会根据[FB_DEVICE]项目的定义生成fbdev设备节点
# 可以有多个[FB_DEVICE], 并生成多个fbdev设备节点
[FB_DEVICE]
# 该fbdev使用的gop(graphic hardware) ID
FB_HWLAYER_ID = 1
# fbdev的framebuffer使用的gop graphic window ID
FB_HWWIN_ID = 0
# deprecated
FB_HWLAYER_DST = 3
# 该fbdev的framebuffer使用的颜色格式
# RGB565 = 1
# ARGB4444 = 2
# ARGB8888 = 5
# ARGB1555 = 6
# YUV422 = 9
# I8 = 4
# I4 = 13
# I2 = 14
FB_HWWIN_FORMAT = 5
# 修改Output color · 0为RGB · 1为YUV
FB_HWLAYER_OUTPUTCOLOR = 1
# 该fbdev的framebuffer的初始化xres
FB_WIDTH = 1280
# 该fbdev的framebuffer的初始化yres
FB_HEIGHT = 720
#在自动获取到当前的显示timing之前 · 使用的初始化gop输出timing 宽
FB_TIMMING_WIDTH = 1920
#在自动获取到当前的显示timing之前 · 使用的初始化gop输出timing 高
FB_TIMMING_HEIGHT = 1080
# 如果系统的mmap有layout项目为E_MMAP_ID_FB
# 那么FBDEV的framebuffer 将使用此处的内存
FB_MMAP_NAME = E_MMAP_ID_FB
# 如果系统的mmap没有为FBDEV layout一块内存
# 那么FBDEV的framebuffer将申请如下长度的内存作为framebuffer
# FB_BUFFER_LEN >= xres*yres*bytesperpixel*displayBufCnt
FB_BUFFER_LEN = 8192
#unit:Kbyte, 4096=4M, fbdev.ko alloc size = FB_BUFFER_LEN*1024

# FBDEV 支持的硬件鼠标配置
[FB_CURSOR]
# 鼠标层使用的gop ID
```

```

FB_HWLAYER_ID = 0
# 鼠标层使用的gop graphic window ID
FB_HWWIN_ID = 0
# deprecated
FB_HWLAYER_DST = 3
# 鼠标层使用的颜色格式
# RGB565 = 1
# ARGB4444 = 2
# ARGB8888 = 5
# ARGB1555 = 6
# YUV422 = 9
# I8 = 4
# I4 = 13
# I2 = 14
FB_HWWIN_FORMAT = 6
# 修改Output color · 0为RGB · 1为YUV
FB_HWLAYER_OUTPUTCOLOR = 1
# 如果系统的mmap有layout项目为E_MMAP_ID_FB
# 那么FBDEV的鼠标层 将使用此处的内存
# 如果系统的mmap没有为FBDEV layout一块内存
# 那么FBDEV的鼠标层将自己申请128K内存
FB_MMAP_NAME = E_MMAP_ID_HW_CURSOR

# deprecated, fbdev 设备之间的z order(谁显示在上层或者下层)
[LAYER_ZORDER]
LAYER_ZORDER0 = 0
LAYER_ZORDER1 = 1
LAYER_ZORDER2 = 2
LAYER_ZORDER3 = 3
LAYER_ZORDER4 = 4

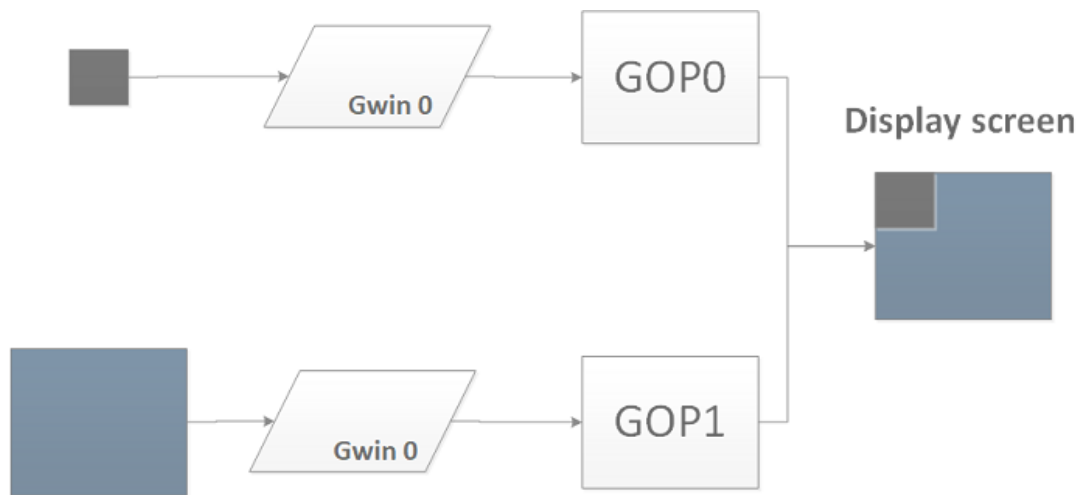
```

4.3. sstar中的图形硬件 (GOP)

观察fbdev.ini 的配置文件，看到每一个fbdev都会绑定 一个GOP ID，以及一个GWIN ID.在sstar的某个硬件平台中，可以有多个GOP，同时单个GOP可能可以控制多个graphic window (Gwin)，每一个Gwin对应一个framebuffer中的display buffer.具体的GOP/Gwin的数量不同的平台存在差异。

- **GOP/Gwin的叠加顺序**

以Takoyaki平台为例，Takoyaki拥有2个GOP，每个gop支持管理1个gwin，如下图所示：



- **GOP之间的差异**

GOP之间可能存在设计差异，不同的GOP的用途存在倾向性。以Takoyaki平台为例

- 支持的颜色格式
- 是否支持stretch

4.4. Takoyaki DISP_GOP 属性

属性	GOP0	GOP1
颜色格式		YUV422
		RGB565
		ARGB8888
	ARGB4444	ARGB4444
	ARGB1555	ARGB1555
	I8	I8
	I4	I4
	I2	I2
缩放	不支持缩放.FB_WIDTH==FB_TIMING_WIDTH · FB_HEIGHT==FB_TIMING_HEIGHT	支持放大，在放大的场景下max(xres)=1280
支持GWIN数	1	1
FB_WIDTH对齐	16/BytesPerPixel	16/BytesPerPixel
alpha0/alpha1 for ARGB1555	支持分别设置alpha0/alpha1	只支持设置alpha1，alpha0=0x0
global alpha	支持	支持
colorkey	支持	支持

4.5 Ikayaki DISP_GOP 属性

属性	GOP0
颜色格式	YUV422
缩放	支持放大,在放大的场景下max(xres)=1280
支持GWIN数	1
global alpha	支持
FB_WIDTH对齐	16/BytesPerPixel
colorkey	支持
alpha0/alpha1 for ARGB1555	只支持设置alpha1,alpha0=0x0
	RGB565
	ARGB8888
	ARGB4444
	ARGB1555
	I8
	I4
	I2

5. Interfaces

简介提到fbdev定义了一些Interfaces给用户操作framebuffer，这里需要注意的是这些接口的实现程度取决于FBDEV的实现者对linux定义的这部分接口的具体实现程度.通常来讲，针对某些具体的平台不需要实现全部的接口定义。

比如:color map/write&read函数等。因为index color显示效果不如ARGB32，操作framebuff使用mmap比wirte&read函数更方便。

除此之外，FBDEV的提供者还可以针对特定平台提供专有的控制接口。

接口支持情况表

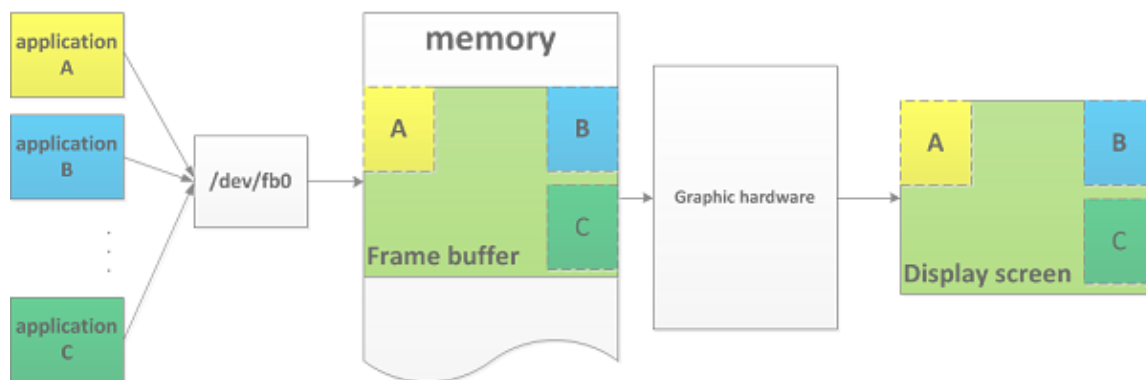
User Space 操作接口	描述	sstar支持平台
open/fopen/close/fclose	以文件方式打开/关闭fbdev	all
write/fwrite/read/fread	以读写文件的方式读写fbdev	none
mmap	将fbdev做内存映射	all
ioctl	fbdev支持ioctl 的方式放送控制命令	all

linux标准iocontrol命令表

iocontrol命令	描述	sstar支持平台
F BIOGET_VSCREENINFO	获取fbdev的可变信息	all
F BIOPUT_VSCREENINFO	设置fbdev的可变信息	all
F BIOGET_FSCREENINFO	获取fbdev的固定信息	all
F BIOPAN_DISPLAY	平移显示，在修改vinfo的可见区域之后	all
F BIOPUTCMAP	设置fbdev的colormap	none
F BIOGETCMAP	获取fbdev的colormap	none
F BIOGET_CON2FBMAP	获取console对应的framebuffer	all
F BIOPUT_CON2FBMAP	映射console对应的framebuffer	all
F BIOBLANK	清空framebuffer	none

sstar专有 iocontrol 命令表

iocontrol命令	描述	sstar支持平台
FBIOGET_SHOW	获取fbdev的显示状态	all
FBIOSET_SHOW	设置fbdev的显示状态	all
FBIOGET_SCREEN_LOCATION	获取显示区域在framebuffer中的位置信息	all
FBIOSET_SCREEN_LOCATION	设置显示区域在framebuffer中的位置信息	all
FBIOGET_GLOBAL_ALPHA	获取fbdev的全局alpha和ARGB1555的index alpha信息	all
FBIOSET_GLOBAL_ALPHA	设置fbdev的全局alpha和ARGB1555的index alpha信息	all
FBIOGET_COLORKEY	获取fbdev的colorkey信息	all
FBIOSET_COLORKEY	设置fbdev的colorkey信息	all
FBIOGET_DISPLAYLAYER_ATTRIBUTES	获取fbdev显示相关信息	all
FBIOSET_DISPLAYLAYER_ATTRIBUTES	设置fbdev显示相关信息	all
FBIOGET_CURSOR_ATTRIBUTE	设置fbdev的鼠标信息	all
FBIOSET_CURSOR_ATTRIBUTE	获取fbdev的鼠标信息	all



Frame buffer layout

